Constructor and Destructor

Constructor

- A constructor is a special member function whose task is to initialize the objects of its class . It is special because its name is the same as the class name. The constructor is invoked when ever an object of its associated class is created. It is called constructor because it construct the values of data members of the class.

Constructor

- Syntax:
  - Class ClassName

```
{
        ClassName(Args)
        {
            Initialization_of_data
        }
};
```

Example:

Example

```
class integer
{
int m,n:
public:
integer(void);                  //constructor declared
------------
};
integer :: integer(void)
{
m=0;
n=0;
}
```

Characteristics Constructor Functions

- They should be declared in the public section .
- They are invoked automatically when the objects are created.
- They don't have return types, not even void and therefore they cannot return values.
- They cannot be inherited , though a derived class can call the base class constructor .
- We can overload constructor, it means we can create more than one constructor of class.
- Like other C++ function , they can have default arguments.
- Constructor can't be virtual.
- Constructor function can be defined inside and outside of the class like other normal member function.

Types of Constructor

- Default Constructor
- Parameterized Constructor
- Copy Constructor

Default Constructor

- Default constructor is also known as zero argument constructors. Default constructor does not have any parameters and is used to set (initialize) class data members. Since, there is no argument used in it, it is called "Zero Argument Constructor".
- In a class, if there is no default constructors defined, then the compiler inserts a default constructor with an empty body in the class in compiled code.
- Example:

```
class Number
{
        int x;
        public:
        Number()
        {
                x=1;
        }
};
```

Example

```
#include <iostream>
using namespace std;

class Number
{
        private:
        int A;
        int B;
        int C;
        public:
        Number();
        void set(int,int,int);
        void print();
};

Number::Number()
{
        A=1;
        B=1;
        C=1;
}

Void Number::set(int x, int y, int z)
{
                A  = x;
                B  = y;
                C  = z;
}

void Number::print()
{
cout<<"Value of A :  "<<A<<endl;
cout<<"Value of B :  "<<B<<endl;
cout<<"Value of C :  "<<C<<endl;
}

int main()
{
        Number obj; //Constructor called when
object created

        obj.print();
        obj.set(10,20,30);
        obj.print();

        return 0;
}
```

Parameterized Constructors

- The constructors that can take arguments are called parameterized constructors. Using parameterized constructor we can initialize the various data elements of different objects with different values when they are created.
- Example:

```
class integer
{
int m,n;
public:
integer( int x, int y);
--------
---------
};
```

Parameterized Constructors

```
integer:: integer (int x, int y)
{
m=x;
n=y;
}
```

- the argument can be passed to the constructor by calling the constructor implicitly and explicitly.

```
integer Int1 = integer(5,100);    // explicit call
integer Int2(6,100);       //implicit call
```

Example

```
#include<iostream.h>
class integer
{
        int m,n;
        public:
        integer(int,int);
        void display(void)
        {
                cout<<"m="<<m<<endl;
                cout<<"n="<<n<<endl;
        }
};
integer :: integer( int x,int y) // constructor defined
{
        m=x;
        n=y;
}
int main( )
{
        integer Int1(5, 100);      // implicit call
        integer Int2=integer(25,75);          //explicit call
        cout<<"\n objectl "<<endl;
        Int1.display();
        cout<<"\n object2: "<<endl;
        Int2.display();
        return 0;
}
```

Constructor function overloading

- Constructors can be overloaded in a similar way as function overloading.
- Overloaded constructors have the same name (name of the class) but the different number of arguments. Depending upon the number and type of arguments passed.
- Example:

```
class Person {
        int age;
        public:
         // 1. Constructor with no arguments
         Person() {
                        age = 20;
         }
        // 2. Constructor with an argument
         Person(int a) {
                        age = a;
         }
};
```

Example

Example:-

```
#include<iostream>
#include<string.h>
using namespace std;
class Student
{
        private:
        char Name[30];
        int age;
        public:
        Student()
        { }
        Student(char Name[], int y);
        void display( )
        {
                cout<<"Name: "<<Name<<endl;
                cout<<"Age: "<<age<<endl;
        }
        void getdata()
        {
                cout<<"Enter Name:";
                cin>>Name;
                cout<<"Enter Age:";
                cin>>age;
        }
};
Student : : Student(char x[], int y)
{
                strcpy(Name,x);
                age=y;
}
int main( )
{
                Student std;
                Student std1=Student("Sagar Pandey",24);
                cout<<"Initialize using getdata function:\n";
                std.getdata();
                cout<<"Givern data from Constructor:\n";
                std1.dispalay();
                cout<<"Givern data from getdata function:\n";
                std.display ();
                return 0;
}
```

Constructor with Default Arguments

- Like functions, it is also possible to declare constructors with default arguments.
- Example:

```
class Integer
{
        int a,b,c;
    public:
      Integer(int x, int y, int z=5)
       {
                a=x;
                b=y;
                c=z;
       }
 }
```

- Here z is the default argument.

Example

```
#include<iostream>
class Number
{
      int X,Y;
   public:

      //Default or no argument constructor.
      Number()
      {
            X = 0;
            Y = 0;

            cout << endl << "Constructor Called";
      }
//Perameterized constructor with default
argument.
      Number(int A, int B=20)
      {
            X = A;
            Y = B;

            cout << endl << "Constructor Called";
      }
      void putValues()
      {
            cout << endl << "Value of X : " << X;
            cout << endl << "Value of Y : " << Y << endl;
      }
};
int main()
{
   Number N1= Number(10);
   cout << endl <<"N1 Value Are : ";
   N1.putValues();
   Number N2;
   cout << endl <<"N2 Value Are : ";
   N2.putValues();
   Number N3(44,77);
   cout << endl <<"N3 Value Are : ";
   N3.putValues();

   return 0;
}
```

Copy Constructor

- A copy constructor is used to declare and initialize an object from another object. It means one object member variable values is assigned to another object member variables.
- Example

  Integer(Integer &in);//Declaration

  Integer int1(int2);//Call statement

Example

```
#include<iostream>
using namespace std;
class code
{
        int id;
        public:
        code ( ) { } //constructor
        code (int a)
        {
                id=a;
        } //constructor
        code(code &x)
        {
                id=x.id;
        }
        void display( )
        {
                cout<<id;
        }
};
int main( )
{
        code A(100);
        code B(A);
        code C=A;
        code D;
        D=A;
        cout<<" \n id of A :";
        A.display( );
        cout<<" \nid of B :";
        B.display();
        cout<<" \n id of C:";
        C.display();
        cout<<" \n id of D:";
        D.display();
        return 0;
}
```

Dynamic Constructor

- The constructors can also be used to allocate memory while creating objects . This will enable the system to allocate the right amount of memory for each object when the objects are not of the same size, thus resulting in the saving of memory.
- Allocate of memory to objects at the time of their construction is known as dynamic constructors of objects. The memory is allocated with the help of new operator.

Example

```
#include<iostream>
#include<string.h>
using namespace std;
class String
{
        char *name;
        int length;
        public:
        String ( )
        {
        length=0;
        name= new char [length+1];
        /* one extra for \0 */
        }
        String(char *s) //constructor 2
        {
                length=strlen(s);
                name=new char [length+1];
                strcpy(name,s);
        }
        void display(void)
        {
                cout<<name<<endl;
        }
        void join(String &a ,String &b)
        {
            length=a.length +b.length;
            name=new char[length+1]; /* dynamic allocation */
            strcpy(name,a.name);
            strcat(name,b.name);
        }
        };
        int main( )
        {
        String name1("Ram"), name2("Kumar"), name3("Budhathoki");
                        String s1,s2;
                        s1.join(name1,name2);
                        s2.join(s1,name3);
                        cout<<"\nFirst Name: ";
                        name1.display();
                        cout<<"\nMiddle Name: ";
                        name2.display();
                        cout<<"\nLast Name: ";
                        name3.display();
                        cout<<"\nFirst and Middle Name: ";
                        s1.display();
                        cout<<"\nFull Name: ";
                        s2.display();
                        return 0;
        }
```

Destructor

- The destructor is used to destroy the objects that have been created by a constructor. Like a constructor, the destructor is a member function and will have exact same name as the class name prefixed with a tilde (~).
- It can neither return a value nor can it take any parameters.
- Destructor is a special member function of a class that is executed whenever an object of it's class goes out of scope or whenever the delete expression is applied to a pointer to the object of that class.
- It will be invoked implicitly by the compiler upon exit from the program to clean up storage that is no longer accessible. It is a good practice to declare destructor in a program since it releases memory space for future use.
- Example:

```
~Integer(){}
```

Characteristics of Destructor

- Name of the destructor is same as the class name. However, destructor name is preceded by the tilde (~) symbol.
- A destructor is called when a object goes out of scope.
- A destructor is also called when the programmer calls delete operator on an object.
- Like a constructor, destructor is also declared in the public section of a class.
- The order of invoking a destructor is the reverse of invoking a constructor.
- Destructors do not take any arguments and hence cannot be overloaded.
- A destructor does not return a value.
- Destructor cannot be inherited.
- Unlike constructors, destructors can be virtual.

Example

```
#include<iostream>
using namespace std;
int count=0;
class alpha
{
        public:
        alpha( )
        {
        count ++;
        cout<<count<<" no of object
        created :"<<endl;
        }
        ~alpha( )
        {
        cout<<count<<" no of object
        destroyed :"<<endl;
                count--;
        }
};
int main( )
{
        cout<<"\nEntered main:\n";
        alpha A1,A2,A3,A4;
        {
        cout<<" \nEntered block 1:\n";
                alpha A5;
        }
        {
        cout<<" \nEntered block2:\n";
                alpha A6;
        }
        cout<<"\nre-entered main:\n";
        return(0);
}
```

More Example

```
#include <iostream>
using namespace std;
class Student
{
        private:
                int age;
        public:
                Student(int n)
                {
                        age= n;
                        cout<<"Memory allocated for student "<<age<<endl;
                }
                ~Student()
                {
                        cout<<"Memory deallocated for student "<<age<<endl;
                }
};
int main()
{
        Student s1(20);
        Student s2(25);
        Student s3(45);
        return 0;
}
```